

MessyMem: Learning-from-Doing Memory for Mobile Manipulation

Anuva Banwasi William Muckelroy III Priya Sundaesan Linfeng Zhao
Jeannette Bohg Cherie Ho

Stanford University

<https://messymem.github.io>

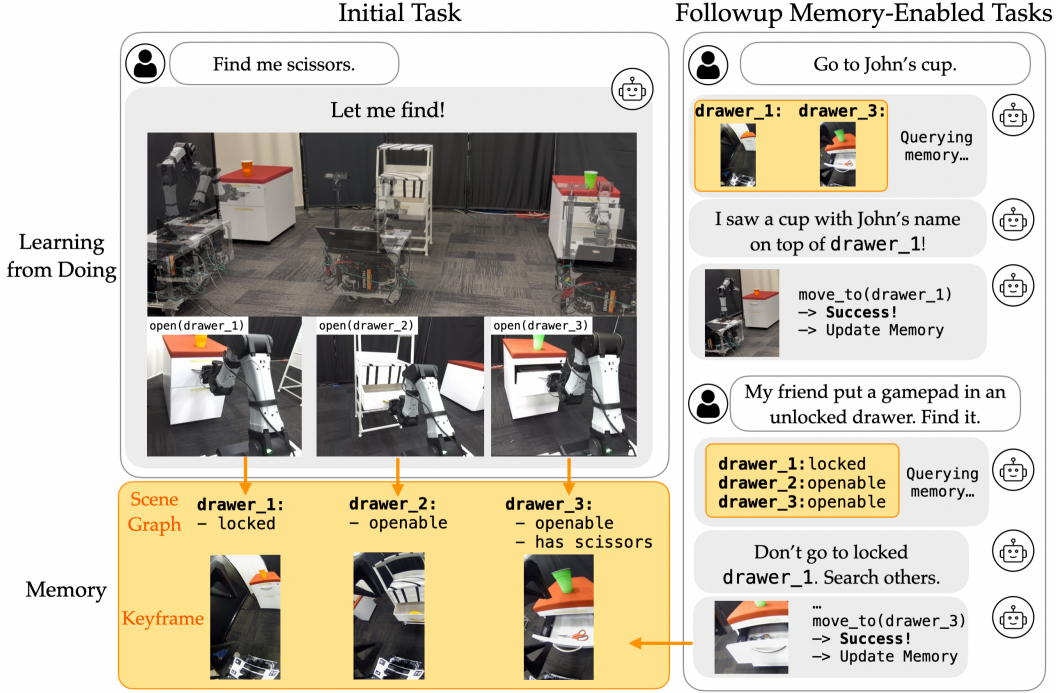


Figure 1: MessyMem is a persistent mobile-manipulation memory system maintaining a 3D scene graph with interaction-derived properties and linked keyframes. **(Left)** The robot builds memory while exploring to *Find me scissors*. **(Right)** On later open-vocabulary tasks, it reuses memory rather than re-exploring, retrieving fine-grained visual evidence to locate *John's cup* and interaction-derived properties to avoid the locked drawer.

Abstract: Mobile manipulators deployed across many rooms and visits should improve with experience: after discovering that a cabinet is locked or finding an object in a drawer, the robot should reuse that knowledge rather than start each task from scratch. Yet today's robots often treat each task as new: compact scene representations omit interaction-derived knowledge, raw video histories are difficult to query, and VLM planners reason at inference time without persistently updating what the robot knows. We present MessyMem, a persistent memory system that enables mobile manipulators to learn from experience and reuse that knowledge across future tasks. It maintains a spatially grounded 3D scene graph of objects and locations, augments it with properties and outcomes learned through interaction, and links visual observations for fine-grained recall. We evaluate MessyMem in simulation and on a real mobile manipulator. In a continuous 25-task simulation spanning over 3 hours, MessyMem achieves 80.0% task progress, outperforming the strongest ablation by 14.8 percentage points and the strongest external baseline by 28.9 points, while retrieving task-relevant evidence from thousands of stored keyframes and over an hour into the past.

Keywords: Interaction-Grounded Memory, Mobile Manipulation, Scene Graph

1 Introduction

Imagine asking a household robot to bring a child their favorite mug. Without any extra context, the robot should be able to recognize it from a dinosaur sticker on the handle, a detail it noticed during yesterday’s cleanup. Later, when asked to make coffee, it should head to the cabinet where the coffee grounds are kept, rather than scanning the whole kitchen. When searching for sugar to go with the coffee, it should remember which cabinets are locked, and only try the unlocked ones. These behaviors share a property current robots lack: a persistent memory of what observation and physical interaction have revealed, anchored across the space the robot works in over a long time. Without such memory, every task starts from scratch, such as re-opening every cabinet.

These examples point to three requirements for memory in mobile manipulation:

- (1) **Globally queryable & spatially grounded.** Memory must support efficient queries across large environments (rooms, floors, buildings) and long deployments (across sessions), in a form compact enough to actually search rather than a sliding window that forgets or an unindexed video log that cannot be queried at scale. Each query must return not only *what* the robot has seen but *where* to go to find it, so the planner can pick a visit order and navigate directly instead of re-exploring.
- (2) **Updatable from interactions.** Many object properties (e.g., weight, emptiness, locked-vs-unlocked, articulation) cannot be inferred from passive observation and must be learned through physical interaction. Memory must learn and save what each interaction reveals.
- (3) **Fine-grained.** For new tasks, text-based summaries alone are often insufficient; the robot must be able to retrieve fine-grained visual context on demand (e.g., the exact shelf where coffee was last seen, the label on a particular container).

The concept of memory has a rich history in robotics, but has yet to deliver on the three above properties simultaneously. 3D scene representations such as scene graphs [1, 2, 3, 4, 5] capture environment structure from passive observation, but miss what a robot learns from physical interaction. Raw video frames preserve fine-grained detail but are expensive to search; keyframe-selection and video-memory methods [6, 7, 8] provide more compact visual histories, but generally lack the persistent spatial grounding needed for mobile manipulation. Recent scene-graph and keyframe systems [9, 10, 11] primarily target navigation. Closest to our setting, CuriousBot [12] and RoboEXP [13] build 3D scene graphs through interactive exploration, but do not pair properties learned through interaction with fine-grained visual evidence that can be retrieved for future tasks.

Our key contribution is MessyMem, a memory system that lets mobile manipulators carry knowledge across tasks, large spaces, and long timescales instead of re-exploring. It combines three aforementioned memory properties that support flexible queries for diverse future tasks (Fig. 1). MessyMem builds a compact 3D scene graph whose nodes are observed objects associated with their world-frame positions, keeping memory **globally queryable and spatially grounded**. After an action, an interaction analyzer infers object properties that are only revealed through the robot’s actions and attaches a summary to the corresponding node, keeping memory **updatable from interactions**. Finally, it saves keyframes and links them to scene-graph nodes, enabling **fine-grained** retrieval of image detail. By indexing both interaction-derived properties and keyframes through the scene graph, MessyMem can retrieve a compact, task-relevant subset of past experience at plan time rather than searching through raw hours-long visual histories.

We evaluate MessyMem in simulation and on a real TidyBot++ platform, including 50-trial controlled evaluations and a 25-task sequence spanning over 3 hours of continuous execution. Across these experiments, MessyMem consistently outperforms its ablations and external memory baselines. On the long-horizon evaluation, it reaches 80.0% task progress, 14.8 percentage points above the strongest ablation and 28.9 points above the strongest external baseline.

2 Related Work

Queryable spatial memory. Metric-semantic maps and 3D scene graphs provide a natural representation for globally queryable and spatially grounded robot memory, from object-centric environment graphs [1, 2, 3, 4] to open-vocabulary maps that support language queries over objects and places [14, 15, 16, 17, 18, 5]. Such maps can ground large-scale LLM planning [19], but are typically constructed from passive observations of geometry, location, and semantics. MessyMem augments this structure with action-revealed information, including object properties such as locked, empty, or heavy, together with execution outcomes and failure reasons.

Fine-grained visual memory. Embodied agents also benefit from retaining image, video, or keyframe histories rather than conditioning only on the current observation [20, 21, 22, 23]. Recent robot-memory systems build multimodal environment memories for embodied interactive agents [24], use 3D scene-memory snapshots for exploration and reasoning [25], retrieve experience keyframes [6], select key history frames for imitation [7], combine short-horizon video with long-horizon text memory [8], or build non-parametric embodied memories for retrieval and generation [26]. These approaches preserve rich visual evidence, but generally do not couple it with persistent spatial structure and interaction-derived state. MessyMem instead links keyframes to persistent scene-graph entities, keeping fine-grained visual evidence spatially grounded for future planning.

Interaction-grounded memory and planning. Planning under partial observability treats hidden state as a belief-update problem, from POMDPs and decision making under uncertainty [27, 28, 29] to robotic task-and-motion planning in belief space [30, 31, 32]. Recent work also uses foundation models as uncertainty estimators for belief-space planning [33]. Interactive-perception systems infer articulation or actionability from physical trials [34, 35].

RoboEXP and CuriousBot are closest to our setting in using interaction to build action-conditioned or relational scene graphs [13, 12]. These systems primarily enrich scene representations during exploration, and RoboEXP uses a predefined relational schema (e.g., inside, on). In contrast, MessyMem updates persistent memory during task execution with open-vocabulary properties and outcomes that the planner can reuse across later tasks. We directly compare against RoboEXP under a shared execution stack.

Planner-facing memory. Foundation-model robot planners reason flexibly from language and vision, including affordance-grounded planning [36], embodied multimodal planning [37], vision-language-action policies [38], long-horizon language-to-motion planning [39], and 3D value-map composition [40]. World-model and latent-memory methods learn implicit state for control [41, 21]. These systems provide strong inference-time reasoning, but do not expose an editable object-level memory that persists across tasks and combines structured state with retrievable visual evidence.

Taken together, prior work provides complementary pieces of spatial structure, visual recall, interaction-derived state, and foundation-model planning. MessyMem combines these in a persistent memory interface that can be accumulated and reused across tasks and long time horizons.

3 Method

We propose MessyMem, a persistent memory system for mobile manipulation (Fig. 2). The robot incrementally builds memory from RGB-D observations and manipulation outcomes. Execution proceeds in a closed loop: retrieve task-relevant memory, plan and execute a primitive, analyze its outcome, update memory, and replan against the updated state.

3.1 Globally Queryable & Spatially Grounded Text Memory: 3D Scene Graph

We maintain a persistent 3D scene graph keyed by object instances. This compact structure serves as the index to which interaction-derived properties (§3.2) and keyframes (§3.3) are attached. A **scene graph builder** constructs the graph incrementally from incoming RGB-D observations. Each image is passed to SAM3 [42] to segment objects from a predefined set of category prompts; each mask

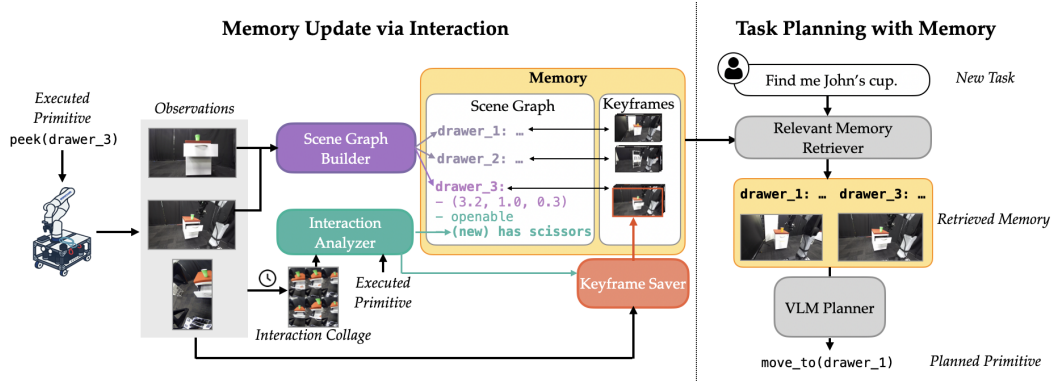


Figure 2: **MessyMem approach overview.** (Left) A scene graph builder incrementally maintains a persistent 3D graph of observed objects and their locations, while a keyframe saver links selected visual observations to the corresponding scene-graph entries. After manipulation, a VLM-based **interaction analyzer** reasons over the interaction collage and writes action-revealed properties, contents, and execution outcomes back to the scene graph. (Right) For each planning step, a **memory retriever** uses the task and scene graph to surface relevant structured properties and linked keyframes. A **VLM planner** combines this retrieved memory with the current observation to select the next primitive; execution then produces new evidence that updates memory for subsequent decisions.

is deprojected to 3D using the depth image, and same-label detections are clustered and associated with existing entries (details in Appendix). Each entry stores an `object_id` (e.g., `drawer_3`), a 3D centroid for spatial grounding, and a property dictionary maintained by the interaction analyzer.

Previously observed objects remain in the graph after leaving the current camera view. Interaction-derived properties and visual keyframes are associated with the same persistent entries, allowing remembered state and visual evidence to remain grounded to locations that the planner can revisit.

3.2 Interaction-Updated Text Memory: VLM Interaction Analyzer

After each manipulation primitive, a **VLM interaction analyzer** converts the corresponding visual interaction window into structured updates to the targeted scene-graph entry. This captures information that passive observation alone may not reveal, including object properties such as whether a cabinet is locked or openable, newly discovered fixture contents, and execution outcomes such as success, failure, or failure reason.

The system collects the corresponding action window, forms a multi-frame collage, and queries the VLM-based analyzer. Reasoning over the interaction window rather than only a single before/after observation exposes how the action unfolds: for example, whether the gripper misses a handle, slips during a pull, or successfully grasps while the cabinet itself remains stationary. The analyzer parses this evidence into structured fields and writes them to the relevant scene-graph entry. It updates a fixed set of broadly useful properties (e.g., `pickable`, `fullness`, `openable`), can add open-vocabulary properties when useful (e.g., `material`, `has_handles`), records execution outcomes and failure reasons, and refreshes fixture contents from newly observed evidence.

Multi-frame context allows the analyzer to distinguish visually similar end states arising from different causes, such as a missed grasp versus a successful grasp on a locked cabinet (Appendix Fig. 4). For example, after `peek(drawer_3)`, the analyzer may write contents: `scissors` to the `drawer_3` node; after an unsuccessful opening attempt, it can distinguish a missed grasp from evidence that the drawer is locked.

New evidence updates the corresponding current scene-graph state. If later observations reveal different contents or object state, the newer value overwrites the stale structured property, allowing outdated or incorrect memory to be corrected. Keyframes, in contrast, retain the underlying visual history rather than being overwritten.

3.3 Fine-Grained Visual Memory: Linked Keyframes

The **keyframe saver** maintains a bank of selected RGB observations linked to the persistent scene-graph entries they describe. Keyframes are saved from both interactions and scene updates. Around each manipulation action, the system stores the frame before the action, the frame after the action, and the frame the interaction analyzer identifies as most informative; these frames are linked to the scene-graph entry targeted by the action. Outside manipulation, frames are also saved when the scene graph registers a new object or updates an existing entry, and are linked to the visible entries. Near-duplicate observations are removed to reduce redundant growth (details in Appendix).

Each keyframe stores a full-resolution RGB image and a downsampled thumbnail together with metadata such as its timestamp, originating action, and linked scene-graph entries. Interaction-derived keyframes can additionally receive short free-text labels from the analyzer (e.g., `spoiled bananas`, `open drawer interior`), providing retrieval cues for visual contents that may not correspond directly to a scene-graph object label.

Together, the scene graph maintains compact object-centric state and spatial grounding, while linked keyframes retain appearance, text, clutter, and other instance-level details from past observations.

3.4 Memory Retrieval and VLM Task Planning

At each planning step, the **memory retriever** first surfaces a compact, task-relevant subset of the accumulated memory. The **VLM planner** then combines this evidence with the current observation and scene graph to select the next primitive.

Given the task description and current scene-graph state, the retriever builds a candidate pool from three complementary sources: (1) keyframes linked to scene-graph entries, (2) keyframes whose analyzer-written labels match content in the task, and (3) recent interaction keyframes that provide short-term context. Candidates from these sources are merged and deduplicated, with per-source caps preventing recently observed entities from overwhelming the retrieval pool.

Retrieval then proceeds in two stages. In the first, text-only stage, a VLM receives a compact catalog describing each candidate by its linked scene-graph entries, labels, originating action, trigger, and other metadata, together with the task and a compact scene-graph summary. It selects a shortlist of frames likely to help solve the task. In the second, multimodal stage, the VLM inspects thumbnails of the shortlisted frames and selects a small set of clear, non-redundant observations for the planner. This staged design uses structured memory to first narrow the candidate set, then applies visual reasoning only to the most relevant frames. Appendix Fig. 5 illustrates an example in which retrieved keyframes preserve an instance-specific cue, the name printed on a cup, that is absent from the structured scene graph.

The VLM planner receives the retrieved full-resolution frames together with short descriptions of why they were selected, the full scene graph as JSON, the current camera observation, the task description, and the available motion primitives. It selects the next primitive and scene-graph target; after execution, the interaction analyzer updates the relevant memory entries, and the next planning step retrieves again against the newly updated state.

4 Experimental Setup

We evaluate MessyMem on memory-intensive scenarios in simulation and the real world.

Evaluation Protocol. *Initialization:* Each trial begins with a predefined trajectory that observes fixture locations (e.g., cabinets, drawers, counters) without revealing their contents; contents become observable only after interaction. *Task sequence:* Tasks are issued sequentially after the previous task completes or fails, requiring information acquired earlier to persist across decisions. *Primitives:* All methods use the same navigation, fixture-interaction, and manipulation primitives, with embodiment-specific details in the Appendix. *Metrics:* We report task progress [8, 6], defined by

task-specific success criteria, together with the number of actions. For the two short simulation scenarios, we run 50 trials per method and report mean task progress with 95% confidence intervals.

Compared Methods. All methods share the same planner, perception stack, and action primitives, isolating the effect of the memory representation.

- **MessyMem (SG+IA+KF):** Our full method, combining a persistent scene graph (SG), interaction-analyzer updates (IA), and linked keyframe memory (KF).
- **SG+IA:** MessyMem without keyframe memory. This tests whether spatially grounded structured memory and interaction-derived properties are sufficient without access to fine-grained visual history.
- **SG+KF:** MessyMem without the interaction analyzer. The method retains object positions and linked keyframes but does not store properties or outcomes inferred from interactions.
- **SG:** A scene graph containing object labels and positions, without interaction-derived updates or keyframe memory. This tests the limit of a globally queryable spatial representation alone.
- **RoboEXP [13]:** We adapt RoboEXP’s Action-Conditioned Scene Graph (ACSG) memory to our execution stack. Its predefined relational representation stores structured spatial and action-conditioned information, while the planner, perception, and primitives remain shared with the other methods.
- **MemER-style [6]:** We implement MemER’s episodic visual-memory paradigm, in which a VLM selects salient observations that are clustered temporally into representative keyframes. The planner reasons from these episodic frames without MessyMem’s persistent scene-graph indexing or interaction-derived state.
- **MemER+Fixtures:** To separate episodic-memory limitations from basic mobile-navigation grounding, we additionally provide MemER-style memory with the identities and positions of visible and out-of-view fixtures.

4.1 Simulation Scenarios

Environment. We instantiate our scenarios in the MuJoCo [43]-based RoboCasa365 simulator [44] for household mobile manipulation. Full scenario definitions and success criteria in Appendix.

Locked-and-Unlocked Cabinets: Globally queryable Updateable from interactions Among three cabinets, one is locked but its status is unknown before interaction. The robot must first *look for a banana* and then *look for ketchup*. Solving the second task requires retaining both cabinet contents and the previously discovered locked status.

Clutter-Aware Pick: Globally queryable Updateable from interactions Fine-grained Across two cabinets, the robot first *finds the mustard* and is then asked to *pick up the ketchup*. Ketchup appears in both cabinets, but only one instance is readily accessible; the other is embedded in surrounding clutter. Correct selection requires recalling fine-grained visual context beyond object identity or presence.

25-task Long Horizon: Globally queryable Updateable from interactions Fine-grained We evaluate a continuous sequence of 25 household tasks in a 10-cabinet kitchen containing more than 60 objects and fixtures. The sequence spans four task families: *finding*, *matching*, *manipulation*, and *memory queries*, and executes for over 3 hours without resetting memory. Tasks require recalling observations from much earlier in the sequence, distinguishing visually similar object instances, reasoning about where household objects belong, and reusing information acquired during prior manipulation. Examples include matching a cup to one observed earlier, identifying a reusable bottle among similar variants, recalling which objects were stored together, and finding a mug placed away earlier.

4.2 Real-World Scenarios

Hardware. We use the TidyBot++ platform [45] with a 6-DoF YAM arm and holonomic mobile base. Two ZED 2 RGB-D cameras support scene-graph construction and 3D object localization: a front-facing base camera and a pole-mounted camera that provides a higher vantage point and

Method	Cluttered Pick Task Prog. $\uparrow$	Locked / Unlocked Task Prog. $\uparrow$	25-task Long Horizon Task Prog. $\uparrow$
MessyMem (Ours)	84.0 [77.0, 90.0]	99.0 [97.0, 100.0]	80.0 [77.4, 82.6]
SG + IA	58.0 [53.0, 63.0]	88.0 [82.0, 94.0]	65.2 [63.5, 67.0]
RoboEXP [13]	53.0 [48.0, 58.0]	70.0 [63.0, 76.0]	51.1 [49.1, 53.1]
SG + KF	51.0 [44.0, 58.0]	43.0 [35.0, 51.0]	47.9 [45.5, 50.4]
SG	49.0 [45.0, 53.0]	50.0 [50.0, 50.0]	49.1 [47.4, 51.0]
MemER-style [6]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	1.8 [1.0, 2.5]
MemER + Fixtures	8.0 [3.0, 13.0]	27.0 [16.0, 39.0]	0.7 [0.3, 1.2]

Table 1: MessyMem **consistently outperforms its ablations and external memory baselines**. Mean task progress [95% CI] over 50 trials per method. The short scenarios isolate fine-grained visual and interaction-derived memory, while Long Horizon evaluates all memory properties jointly over 25 consecutive tasks.

observations to the VLM planner. A wrist-mounted Arducam fisheye provides additional views for keyframe memory, while wall-mounted fiducial markers support localization.

Office Drawer Search: Globally queryable Updateable from interactions Fine-grained Across three office drawers, one of which is locked, the robot sequentially searches for scissors, identifies John’s cup, and searches for a newly placed game controller in an unlocked drawer. The sequence requires persistent spatial grounding, reuse of the discovered lock status, and fine-grained visual recall to distinguish objects using details that may not be captured in the scene graph.

Sock Pairing: Globally queryable Updateable from interactions Fine-grained With two sock-filled drawers and one unpaired sock on a chair, the robot first inspects both drawers and then places the loose sock into the drawer containing its matching pair. Since candidate socks can share coarse attributes such as color, success depends on recalling fine-grained appearance, including pattern, while retaining the spatial association between the relevant visual memory and its drawer.

5 Experimental Results

5.1 Simulation Results

Table 1 shows that MessyMem achieves the highest task progress across all three simulated evaluations. No individual memory component suffices across scenarios: interaction-derived state is particularly useful when the robot must reuse properties learned through action, while linked keyframes preserve visual information that is difficult to capture in a compact structured representation. Combining both with a persistent spatial scene graph yields the strongest performance across tasks.

Fine-grained visual memory enables instance-level reasoning. On *Cluttered Pick*, MessyMem reaches **84%** task progress, compared with **58%** for SG+IA and **51%** for SG+KF. Without keyframes, the robot can remember where ketchup was observed but cannot recover the detailed visual arrangement needed to select the less obstructed instance. The full method combines structured grounding with linked keyframes that recover this task-relevant visual evidence.

Interaction-derived memory captures action outcomes. On *Locked / Unlocked*, MessyMem reaches **99%** progress and SG+IA **88%**, compared with **43%** for SG+KF. A saved image alone may not distinguish a locked cabinet from a failed grasp. The Interaction Analyzer reasons over a higher-frequency collage spanning the manipulation, allowing it to determine success or failure, infer properties revealed by the action, and store the resulting state in the scene graph for later tasks.

Comparison with external memory systems. RoboEXP reaches **53%/70%** on Cluttered Pick and Locked / Unlocked, compared with MessyMem at **84%/99%**. Its predefined Action-Conditioned Scene Graph captures spatial structure but not fine-grained appearance or richer interaction-derived

Method	1: “Find Scissors”			2: “Move to John’s cup”			3: “Find placed controller”			Overall		
	Task Prog.↑	Steps↓	SPL↑	Task Prog.↑	Steps↓	SPL↑	Task Prog.↑	Steps↓	SPL↑	Task Prog.↑	Steps↓	SPL↑
MessyMem (Ours)	1.00	6.4	0.85	1.00	1.2	0.90	1.00	3.2	1.00	1.00	10.8	0.92
SG + IA	1.00	6.6	0.90	0.60	1.4	0.60	1.00	3.0	0.96	0.87	11.0	0.82
SG + KF	0.80	7.4	0.67	0.60	3.6	0.34	0.40	6.0	0.19	0.60	17.0	0.40
SG	0.00	9.4	0.00	0.20	1.0	0.20	1.00	5.0	0.81	0.40	15.4	0.34

Table 2: **Per-task breakdown for *Office Drawer Search*.** Task progress, executed steps, and shortest path lengths (SPL) are reported for each sequential goal; overall steps are summed across the full trial ($n = 5$). SPL is success-weighted by path length computed per trial and averaged; this uses a human-annotated optimal number of steps ($L = 6, 1, 4$ for tasks 1, 2, and 3, respectively).

state (e.g., a locked cabinet). MessyMem instead stores action-revealed properties and outcomes while linking keyframes for instance-level visual recall.

We find that MemER-style memory struggles in mobile manipulation, where relevant fixtures leave view and keyframes alone lack persistent spatial grounding. Providing visible and out-of-view fixture identities and positions improves MemER+Fixtures from **0%** to **8%/27%** on the two short scenarios but performance remains low.

5.2 Long-Horizon Memory

The 25-task Long Horizon evaluation tests how these differences compound over extended operation. Across more than 3 hours, MessyMem achieves **80.0%** task progress, outperforming the strongest ablation, SG+IA, by **14.8 percentage points** and the strongest external baseline, RoboEXP, by **28.9 points**. All six paired comparisons remain significant after Holm correction ($p_{\text{Holm}} < 10^{-4}$). The ablations show that neither structured nor visual memory alone is sufficient across a diverse task sequence. SG+IA reaches **65.2%**: interaction-updated structured state supports many tasks but cannot recover visual details absent from the graph, limiting matching and instance-level recall. SG+KF (**47.9%**) retains visual history but cannot persist what prior actions revealed, while SG alone (**49.1%**) lacks both capabilities. MessyMem combines them, using compact structured state when sufficient and retrieving linked visual evidence when finer detail is required.

RoboEXP reaches **51.1%** on Long Horizon, exposing the limitation of predefined structured relations for instance-level distinctions. For example, remembering that a `cereal_box` is inside a cabinet does not distinguish Froot Loops from a visually similar Trix box; MessyMem can instead retrieve linked keyframes to recover the needed visual evidence. MemER-style and MemER+Fixtures reach only **1.8%** and **0.7%**, respectively. Over 3+ hours, MemER’s recency bias makes older evidence increasingly difficult to surface, whereas MessyMem retrieves task-relevant keyframes using the current goal and scene-graph associations.

Memory remains useful as it grows. MessyMem accumulates a median of **9,669 keyframes** with a median retrieval latency of **8.6 s**. The current goal and scene-graph associations narrow thousands of observations to a small candidate set before visual reranking, enabling a median lookback of **62.7 minutes**. Task-relevant experience remains accessible over hours of operation.

5.3 Real-World Results

The simulation findings transfer to the real robot: MessyMem achieves **1.00** task progress in both scenarios, while each ablation exposes a distinct failure mode under real-world variability.

Reusing interaction outcomes and visual evidence. In *Office Drawer Search*, MessyMem is the only method to complete all three sequential goals, reaching **1.00** overall progress in **10.8** steps on average while following closest to the optimal path with an SPL of **0.92**; $n = 5$ trials (Table 2). During drawer exploration, IA-enabled methods can reuse the discovered lock status rather than repeatedly attempting an infeasible interaction; MessyMem completes the first goal in **6.4** steps, compared with 7.4 for SG+KF and 9.4 for SG. The second goal, “Move to John’s cup,” additionally requires

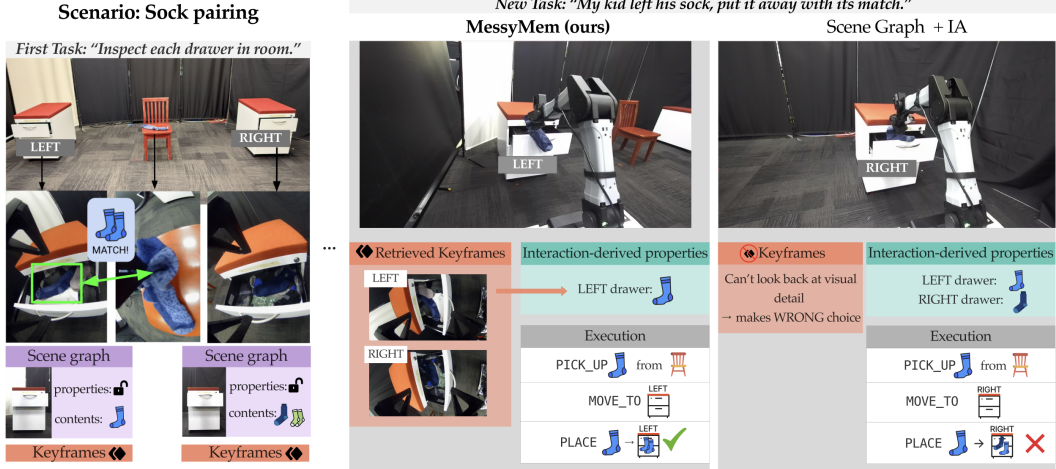


Figure 3: **Sock Matching scenario.** During inspection, *MessyMem* links keyframes of the left and right drawer interiors to its scene graph. When asked to match the loose sock, the VLM planner retrieves these fine-grained observations, compares them against the target sock, and returns to the corresponding drawer. Without keyframes, *SG+IA* cannot recall the needed visual detail and selects the wrong drawer.

fine-grained visual information absent from the scene graph: *MessyMem* reaches **1.00** progress, while *SG+IA* reaches **0.60**. Across the full sequence, methods lacking either interaction-updated state or fine-grained visual recall fail more often and require more actions.

Fine-grained recall for visual matching. The *Sock Pairing* scenario isolates fine-grained visual recall in a spatially grounded setting: after inspecting two drawers, the robot must match a loose sock to a previously observed instance that differs from other candidates primarily in appearance. *MessyMem* retrieves the linked drawer keyframes and compares them against the target sock, yielding **1.00** task progress versus **0.67** for *SG+IA* and *SG+KF* and **0.50** for *SG* over $n = 6$ trials (Fig. 3).

6 Conclusion

We present *MessyMem*, a persistent memory system that enables mobile manipulators to learn from interaction and reuse experience across tasks, large spaces, and long timescales. *MessyMem* combines a spatially grounded 3D scene graph, interaction-derived object properties and outcomes, and linked keyframes that preserve fine-grained visual evidence, yielding memory that is simultaneously **globally queryable**, **updateable from interaction**, and **fine-grained**. Across 50-trial simulation evaluations, a 25-task sequence spanning over 3 hours of continuous execution, and real-world robot experiments, *MessyMem* consistently outperforms its ablations and external memory baselines. On the long-horizon evaluation, it reaches 80.0% task progress, 14.8 percentage points above the strongest ablation and 28.9 points above the strongest external baseline, while retrieving useful evidence from thousands of stored keyframes and over an hour into the past. These results show that structured state, interaction-derived updates, and accessible visual history provide complementary capabilities for persistent robot memory. *MessyMem* is a step toward robots that can accumulate, recall, and act on experience throughout extended operation in complex real-world environments.

Limitations. Our current implementation uses a predefined closed-set prompt for object detections. A natural extension is to incorporate open-vocabulary detectors so the memory can capture more diverse objects and attributes. Our interaction analyzer is also focused on robot-initiated actions; extending it to reason over human actions would broaden the framework to HRI settings where memory must be updated from both robot and human activity. Future work could also extend the same memory-conditioned interface to richer policy primitives, including VLA-based policies.

Acknowledgments

This work is supported by the Toyota Research Institute. This work used Marlowe [46], Stanford University’s GPU-based Computational Instrument, supported by Stanford HAI and Stanford Research Computing. Priya Sundaresan is supported by a NSF GRFP. Cherie Ho is supported by the Croucher Postdoctoral Fellowship. Thank you to Satvik Sharma and Francis Engelmann for helpful discussions. We thank Alberta Longhini, Tyler Lum, and Carlota Parés-Morlans for feedback on earlier drafts of the paper. The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies, either expressed or implied, of the sponsors.

References

- [1] N. Hughes, Y. Chang, and L. Carlone. Hydra: A real-time spatial perception system for 3D scene graph construction and optimization. In *Robotics: Science and Systems (RSS)*, 2022.
- [2] A. Rosinol and L. Carlone. Kimera: an open-source library for real-time metric-semantic localization and mapping. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2021.
- [3] I. Armeni, Z.-Y. He, J. Gwak, A. R. Zamir, M. Fischer, J. Malik, and S. Savarese. 3d scene graph: A structure for unified semantics, 3d space, and camera. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 5664–5673, 2019.
- [4] J. McCormac et al. Semanticfusion: Dense 3d semantic mapping with convolutional neural networks. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [5] Q. Gu, A. Kuwajerwala, S. Morin, K. M. Jatavallabhula, B. Sen, A. Agarwal, C. Rivera, W. Paul, K. Ellis, R. Chellappa, C. Gan, C. M. de Melo, J. B. Tenenbaum, A. Torralba, F. Shkurti, and L. Paull. Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning. *arXiv preprint arXiv:2309.16650*, 2023.
- [6] A. Sridhar et al. Memer: Scaling up memory for robot control via experience retrieval. *arXiv preprint arXiv:2510.20328*, 2025.
- [7] M. S. Mark, J. Liang, M. Attarian, C. Fu, D. Dwibedi, D. Shah, and A. Kumar. Bpp: Long-context robot imitation learning by focusing on key history frames. *arXiv preprint arXiv:2602.15010*, 2026.
- [8] M. Torne, K. Pertsch, H. Walke, K. Vedder, S. Nair, B. Ichter, A. Z. Ren, H. Wang, J. Tang, K. Stachowicz, K. Dhabalia, M. Equi, Q. Vuong, J. T. Springenberg, S. Levine, C. Finn, and D. Driess. Mem: Multi-scale embodied memory for vision language action models. *arXiv preprint arXiv:2603.03596*, 2026.
- [9] H. Yin, X. Xu, Z. Wu, J. Zhou, and J. Lu. Sg-nav: Online 3d scene graph prompting for llm-based zero-shot object navigation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [10] A. Werby, D. Rotondi, F. Scaparro, and K. O. Arras. Keysg: Hierarchical keyframe-based 3d scene graphs. *arXiv preprint arXiv:2510.01049*, 2025.
- [11] T. Shan, A. Rajvanshi, N. Mithun, and H.-P. Chiu. Graph2nav: 3d object-relation graph generation to robot navigation. *arXiv preprint arXiv:2504.16782*, 2025.
- [12] Y. Wang, L. Feroselle, T. Kelestemur, J. Wang, and Y. Li. Curiousbot: Interactive mobile exploration via actionable 3d relational object graph. *arXiv preprint arXiv:2501.13338*, 2025.
- [13] H. Jiang et al. Roboexp: Action-conditioned scene graph via interactive exploration for robotic manipulation. *arXiv preprint arXiv:2402.15487*, 2024.

- [14] N. M. M. Shafiullah, C. Paxton, L. Pinto, S. Chintala, and A. Szlam. Clip-fields: Weakly supervised semantic fields for robotic memory. *arXiv preprint arXiv:2210.05663*, 2022.
- [15] C. Huang, O. Mees, A. Zeng, and W. Burgard. Visual language maps for robot navigation. *arXiv preprint arXiv:2210.05714*, 2022.
- [16] K. M. Jatavallabhula, A. Kuwajerwala, Q. Gu, M. Omama, T. Chen, A. Maalouf, S. Li, G. Iyer, S. Saryazdi, N. Keetha, A. Tewari, J. B. Tenenbaum, C. M. de Melo, M. Krishna, L. Paull, F. Shkurti, and A. Torralba. Conceptfusion: Open-set multimodal 3d mapping. In *Robotics: Science and Systems (RSS)*, 2023.
- [17] J. Kerr, C. M. Kim, K. Goldberg, A. Kanazawa, and M. Tancik. Lerf: Language embedded radiance fields. *arXiv preprint arXiv:2303.09553*, 2023.
- [18] S. Peng, K. Genova, C. Jiang, A. Tagliasacchi, M. Pollefeys, and T. Funkhouser. Openscene: 3d scene understanding with open vocabularies. *arXiv preprint arXiv:2211.15654*, 2022.
- [19] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Suenderhauf. Sayplan: Grounding large language models using 3d scene graphs for scalable robot task planning. *arXiv preprint arXiv:2307.06135*, 2023.
- [20] M. Savva et al. Habitat: A platform for embodied ai research. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [21] E. Parisotto et al. Neural map: Structured memory for deep reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2018.
- [22] E. Parisotto and R. Salakhutdinov. Stabilizing transformers for reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2020.
- [23] B. Baker et al. Video pretraining (vpt): Learning to act by watching unlabeled online videos. *arXiv preprint arXiv:2206.11795*, 2022.
- [24] Y. Liu, X. Song, K. Jiang, W. Chen, J. Luo, G. Li, and L. Lin. Meia: Multimodal embodied perception and interaction in unknown environments. *arXiv preprint arXiv:2402.00290*, 2024.
- [25] Y. Yang, H. Yang, J. Zhou, P. Chen, H. Zhang, Y. Du, and C. Gan. 3d-mem: 3d scene memory for embodied exploration and reasoning. *arXiv preprint arXiv:2411.17735*, 2024.
- [26] Q. Xie, S. Y. Min, P. Ji, Y. Yang, T. Zhang, K. Xu, A. Bajaj, R. Salakhutdinov, M. Johnson-Roberson, and Y. Bisk. Embodied-rag: General non-parametric embodied memory for retrieval and generation. *arXiv preprint arXiv:2409.18313*, 2024.
- [27] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998. doi:10.1016/S0004-3702(98)00023-X.
- [28] M. J. Kochenderfer. *Decision Making Under Uncertainty: Theory and Application*. MIT Press, Cambridge, MA, 2015.
- [29] M. J. Kochenderfer, T. A. Wheeler, and K. H. Wray. *Algorithms for Decision Making*. MIT Press, Cambridge, MA, 2022.
- [30] L. P. Kaelbling and T. Lozano-Perez. Unifying perception, estimation and action for mobile manipulation via belief space planning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 2952–2959, 2012. doi:10.1109/ICRA.2012.6225237.
- [31] L. P. Kaelbling and T. Lozano-Perez. Integrated task and motion planning in belief space. *The International Journal of Robotics Research*, 32(9–10):1194–1227, 2013. doi:10.1177/0278364913484072.

- [32] C. R. Garrett, C. Paxton, T. Lozano-Perez, L. P. Kaelbling, and D. Fox. Online replanning in belief space for partially observable task and motion problems. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 5678–5684, 2020.
- [33] L. Zhao, W. McClinton, A. Curtis, N. Kumar, T. Silver, L. P. Kaelbling, and L. L. S. Wong. Seeing is believing: Belief-space planning with foundation models as uncertainty estimators. *arXiv preprint arXiv:2504.03245*, 2025.
- [34] K. Mo, L. Guibas, M. Mukadam, A. Gupta, and S. Tulsiani. Where2act: From pixels to actions for articulated 3d objects. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [35] Z. Jiang, C.-C. Hsu, and Y. Zhu. Ditto: Building digital twins of articulated objects from interaction. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [36] M. Ahn et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- [37] D. Driess et al. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*, 2023.
- [38] A. Brohan et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- [39] K. Lin, C. Agia, T. Migimatsu, M. Pavone, and J. Bohg. Text2motion: From natural language instructions to feasible plans. *arXiv preprint arXiv:2303.12153*, 2023.
- [40] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. *arXiv preprint arXiv:2307.05973*, 2023.
- [41] D. Hafner et al. Dream to control: Learning behaviors by latent imagination. *International Conference on Learning Representations (ICLR)*, 2020.
- [42] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris, C. Ryali, K. V. Alwala, H. Khedr, A. Huang, J. Lei, T. Ma, B. Guo, A. Kalla, M. Marks, J. Greer, M. Wang, P. Sun, R. Rädle, T. Afouras, E. Mavroudi, K. Xu, T.-H. Wu, Y. Zhou, L. Momeni, R. Hazra, S. Ding, S. Vaze, F. Porcher, F. Li, S. Li, A. Kamath, H. K. Cheng, P. Dollár, N. Ravi, K. Saenko, P. Zhang, and C. Feichtenhofer. Sam 3: Segment anything with concepts, 2025. URL <https://arxiv.org/abs/2511.16719>.
- [43] E. Todorov, T. Erez, and Y. Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012. doi:10.1109/IROS.2012.6386109.
- [44] S. Nasiriany, S. Nasiriany, A. Maddukuri, and Y. Zhu. Robocasa365: A large-scale simulation framework for training and benchmarking generalist robots. In *International Conference on Learning Representations (ICLR)*, 2026.
- [45] J. Wu, W. Chong, R. Holmberg, A. Prasad, Y. Gao, O. Khatib, S. Song, S. Rusinkiewicz, and J. Bohg. Tidybot++: An open-source holonomic mobile manipulator for robot learning, 2024. URL <https://arxiv.org/abs/2412.10447>.
- [46] C. Kapfer, K. Stine, B. Narasimhan, C. Mentzel, and E. Candès. Marlowe: Stanford’s gpu-based computational instrument, 2025. URL <https://doi.org/10.5281/zenodo.14751899>.
- [47] S. Schaal. Dynamic movement primitives: A framework for motor control in humans and humanoid robotics. In H. Kimura, K. Tsuchiya, and A. Ishiguro, editors, *Adaptive Motion of Animals and Machines*, pages 261–280. Springer, Tokyo, Japan, 2006. doi:10.1007/4-431-31381-8.23.

- [48] P. Sundaresan, R. Malhotra, P. Miao, J. Yang, J. Wu, H. Hu, R. Antonova, F. Engelmann, D. Sadigh, and J. Bohg. Homer: Learning in-the-wild mobile manipulation via hybrid imitation and whole-body control, 2025. URL <https://arxiv.org/abs/2506.01185>.
- [49] J. Yang, H. Zhang, F. Li, X. Zou, C. Li, and J. Gao. Set-of-mark prompting unleashes extraordinary visual grounding in gpt-4v, 2023. URL <https://arxiv.org/abs/2310.11441>.

Appendix

This appendix provides additional implementation details, full prompts, memory structure, and scenario descriptions for MessyMem.

A Implementation Details

A.1 Experimental Setup

Each trial consists of a sequence of natural-language instructions revealed one at a time. This requires the robot to reuse information gathered from earlier observations and interactions rather than solving each instruction from scratch. At each planner call, the robot receives the current scene graph, any retrieved keyframes, and the available primitive skills. It selects a high-level action, executes the first step, updates memory when applicable, and then replans from the updated state.

In simulation, tasks are evaluated with a fixed budget of twelve planner calls. A task is successful if the requested instruction is completed within this budget. For sequential tasks, we report the fraction of subtasks completed and the number of high-level steps used.

Simulation. We evaluate in RoboCasa365 [44] kitchen environments with custom memory-intensive task families. The simulated robot is a mobile manipulator with a Franka Panda arm and an Omron mobile base. RGB-D observations are rendered from multiple third-person cameras for scene graph construction, while the wrist camera is used for interaction analysis and keyframes. In scenarios that require prior exploration, the robot first observes or interacts with relevant fixtures to seed memory before receiving the test instruction.

Task success is evaluated from environment state and execution logs rather than from the planner’s self-report. Depending on the scenario, success may require opening the correct fixture, discovering the requested object, placing an object in the correct location, terminating when the goal is already satisfied, or correctly answering a recall-style instruction from memory.

Real world. Our real-world implementation runs on a mobile manipulation platform with a 6-DoF arm, parallel-jaw gripper, RGB-D perception cameras, and a wrist camera. The system maintains an object-centric scene graph online during execution. Perception updates add or refresh objects in the graph, while interaction updates add discovered contents, object properties, task outcomes, and linked keyframes. The real-world tasks are operator-gated for safety, and final task success is operator-confirmed. At the start of each trial the robot’s base orientation is initialized to face forward toward the workspace.

A.2 High-Level Primitive Skills

All methods share the same high-level primitive interface. The low-level controllers differ between simulation and the real robot, but each primitive exposes the same kind of outcome information to the planner and, when enabled, to the interaction analyzer.

Simulation primitives. In simulation, navigation and inspection are implemented with scripted helpers, while door manipulation uses closed-loop policies when available.

- `move_base_to(fixture / pos)`: move the base to a fixture or explicit target pose.
- `move_arm_to(pos)`: move the end effector to a target Cartesian position.
- `OpenCabinet(fixture)`: Opens a target fixture. The primitive automatically opens the fixture closest to the robot’s current location. To capture the dynamic interaction in the Locked/Unlocked Cabinet case, we train a learned policy to open the cabinet.
- `PickPlaceCabinetToCounter(object)`: pick an object from the cabinet and place it onto a counter. Ground-truth object location is used for the primitive.

- `Inspect(fixture, object)`: move the wrist camera through an open fixture to observe its contents.

After each non-motion primitive, the interaction analyzer can inspect the action window and write a success/failure outcome into memory. Motion primitives instead report whether they reached the requested tolerance.

Real-world primitives. On the real robot, manipulation skills are implemented as dynamic movement primitives (DMPs) [47] fit from a small number of teleoperated demonstrations. These include navigation to a selected location, opening and closing drawers, peeking into a fixture, picking up an object, placing an object, and moving to a remembered location. For object-directed manipulation, we use a 3D pointing module following the procedure in Sundaresan et al. [48]. Given the current camera image, a VLM points to a task-relevant grasp location, such as a drawer handle or the body of the target object. This 2D point is back-projected through the aligned depth map into a 3D pre-grasp target in the robot frame, which shifts the demonstrated DMP rollout to the current object location. This lets a single demonstration generalize across nearby object placements. Primitive execution is operator-gated for safety. Interaction-based outcomes such as discovered contents or inferred fixture locked state are updated by the interaction analyzer.

A.3 Scene Graph Construction

SAM3 prompts. We run SAM3 [42] with a scenario-specific prompt set shared across methods, so differences in performance come from the memory system rather than perception vocabulary. The object vocabulary is derived from the task setting, while fixture prompts cover the containers and surfaces relevant for mobile manipulation. For real-world experiments, several object categories (e.g., gaming controller, scissors) present in the experiments are intentionally left out to illustrate the limitation of a closed-set scene graph.

Group	SAM3 prompts
Objects (sim)	categories present in the scenario (e.g., ketchup, mayonnaise, mustard, cereal, spaghetti box, lemon, pear, ...)
Fixtures (sim, perception-fixture mode)	cabinet, drawer, counter, oven, microwave, sink, refrigerator, dishwasher, stove
Real-world objects	drawer, cup, chair, sock

Table 3: SAM3 prompts used for scene graph construction.

Node association and update. Detections with the same semantic label and nearby 3D positions are merged into object-centric scene graph nodes. New observations are associated to existing same-label nodes using spatial proximity, and matched nodes update their centroid with a running average. Detections that do not match any existing node initialize new graph nodes. Object identities reset between trials.

Node format. Scene graph nodes are initialized with spatial and semantic fields. Interaction-derived properties, contents, and task history are attached as the robot acts in the environment. Keyframes are linked to nodes through a separate keyframe index.

```
{
  "cabinet_3": {
    "label": "cabinet",
    "pos": [x, y, z],
    "first_seen_frame": 124,
    "bbox_3d": [...],
    "properties": {
      "accessible": {"value": "unknown", "confidence": 0.5, "source":
"perception"},
```

```

    "pickable": {"value": "unknown", "confidence": 0.5, "source":
"perception"},
    "pushable": {"value": "unknown", "confidence": 0.5, "source":
"perception"},
    "fullness": {"value": "unknown", "confidence": 0.3, "source":
"perception"},
    "openable": {"value": true, "confidence": 0.9, "source": "interaction",
"is_opened": true, "reasoning": "Door visibly rotates open."}
  },
  "contents": [{"name": "banana", "value": "...", "reasoning": "..."}],
  "contents_empty": false,
  "contents_source_action": "Inspect",
  "task_history": [{"task": "OpenCabinet", "success": true}]
}

```

B Prompting and Memory Update Details

B.1 VLM Interaction Analyzer

After each manipulation primitive, the interaction analyzer receives a compact visual summary of the action window, together with the action name, target node, current task, and the node’s prior memory state. The analyzer updates memory only when the visual evidence supports a change, such as a fixture opening, an object being lifted, or contents becoming visible.

Collage construction. For each interaction, we provide the analyzer with frames from before, during, and after the action. The before and after images preserve high-resolution endpoint evidence, while the action collage summarizes the gripper engagement window. This allows the analyzer to distinguish transient execution issues, such as a missed grasp, from persistent object properties, such as a locked drawer.

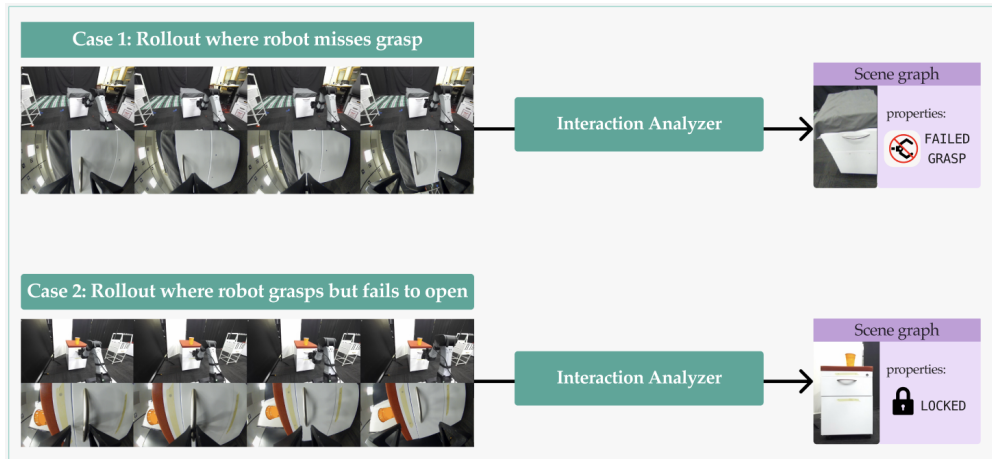


Figure 4: **VLM Interaction Analyzer.** The interaction analyzer reasons over a collage of frames collected during the interaction, capturing dynamic cues that are only visible as the action is taken. MessyMem’s interaction analyzer is able to differentiate between a failed grasp in Case 1, where the robot misses the handle, and a locked cabinet in Case 2, where the robot grasps and pulls but the drawer does not open.

Interaction analyzer prompt. The full interaction analyzer prompt is shown below. The prompt instructs the VLM to reason over the full action window, separate execution failure from object properties, update only with visual support, and return a strict JSON object.

You are an object belief updater for a mobile manipulation robot.

Current belief state of the target object:
{json belief state of the target scene-graph node}

The robot attempted the action: "{action_name}" targeting position {target_pos}.

Terminology: a "fixture" is any scene element with an interior that can be opened to reveal contents.

You will receive THREE images: (1) full-resolution BEFORE (arm at rest, not a grasp-attempt frame); (2) full-resolution AFTER (arm auto-reset to rest - the withdrawal is by design and is NOT evidence of a failed grasp; compare with BEFORE for fixture angle change); (3) a multi-camera collage of the MANIPULATION WINDOW only (pre-position approach and auto-reset excluded). Rows show different camera views of the same window at the same timesteps {rows}; columns are time, left-to-right, labeled BEFORE..AFTER. Use column-wise agreement across rows to disambiguate (e.g. the wrist close-up confirms whether the gripper engaged the handle visible in the base view).

Use the action name to pick the analysis: MANIPULATION (BEFORE/AFTER bracket the interaction; compare to detect what moved/opened/closed/displaced, and report any revealed interior) vs INSPECTION (BEFORE/AFTER are different wrist poses of the same interior; aggregate to identify contents; apparent motion is parallax). Interiors can be dark - check the full-resolution images.

[Nearby fixture nodes in the scene graph, with exact IDs to use in contents_updates and any prior contents listed as a hypothesis to merge.]

Properties of interest (when the action moves/manipulates the target):

- accessible (true/false): can the robot reach it?
- pickable (true/false): only true if FULLY LIFTED off surface.
- pushable (true/false): can it be pushed/slid to a new position?
- fullness (empty / partially_full / full / unknown)
- openable (true/false), distinguishing three cases:
 - (a) opened (any angular change) -> openable=true, is_opened=true
 - (b) no motion despite a genuine attempt -> describe as locked/stuck so the planner moves to a different fixture
 - (c) the attempt failed (no firm contact) -> describe as failed grasp so the planner can retry

Procedure to separate (b) from (c): scan the WRIST row across ALL columns - a close-up of the handle/drawer face means contact -> (b). Else scan SIDE/FRONT rows for jaws CLOSED ON the handle -> (b). Only if BOTH find zero contact evidence -> (c). If ambiguous, PREFER (b). A locked drawer can shear the jaws sideways off the handle, so side-view offset is not evidence of (c) - check the wrist row at the same column first.

Analysis: (1) PHYSICAL CHANGE - track the target across collage frames: shifting or stationary despite contact? Update accessible/pickable/pushable/fullness as warranted (inspections: expect no real change). (2) VISIBLE CONTENTS - identify objects visibly INSIDE any revealed fixture interior, labeled as specifically as the evidence allows; one contents_updates entry per fixture using the exact SG id; empty list if no interior is visible.

Task success (by intent): push/slide -> target visibly displaced (pushable=true); pick/lift -> target FULLY LIFTED; open/close -> fixture in requested state in AFTER (set openable, is_opened); inspection -> a usable view was obtained.

Rules: only update with clear visual evidence; attribute contents only to the fixture they are visibly inside; [] when no interior is visible; one short reasoning per update; do not hallucinate properties or contents.

Return STRICT JSON ONLY: { ... output schema shown below ... }

Property updater and outcome schema. The analyzer writes structured memory updates back to the scene graph. These include task success, failure reason, property updates, newly discovered open-vocabulary properties, and fixture contents. The seeded property vocabulary includes accessible, pickable, pushable, fullness, and openable. The `is_opened` field is used only for openable updates.

```
{
  "task_success": true,
  "failure_reason": null,
  "property_updates": [
    {
      "object_id": "cabinet_3",
      "property": "openable",
      "old_value": "unknown",
      "new_value": true,
      "confidence": 0.9,
      "reasoning": "Door rotates open.",
      "is_opened": true
    }
  ],
  "new_properties": [
    {
      "name": "has_handle",
      "value": true,
      "reasoning": "Pull handle visible."
    }
  ],
  "contents_updates": [
    {
      "fixture_id": "cabinet_3",
      "contents": [
        {
          "name": "banana",
          "description": "..."
        },
        {
          "name": "mug",
          "description": "..."
        }
      ],
      "is_empty": false,
      "reasoning": "Opened cabinet reveals a banana and mug."
    }
  ],
  "confidence": 0.8
}
```

Failed grasp vs. locked fixture. A key role of the analyzer is to separate execution failures from persistent object properties. As shown in Fig. 4, a missed or slipped grasp is recorded as an execution failure, allowing the planner to retry. In contrast, a firm pull with no motion is recorded as evidence that the fixture is locked or stuck, allowing the planner to avoid repeating the same failed interaction.

B.2 Keyframe Memory

Keyframes preserve visual details that are difficult to compress into scene graph fields, such as clutter, object arrangement, and visual distinctions between similar objects. They are linked to scene graph nodes through a keyframe index, allowing later tasks to retrieve visual evidence associated with relevant objects, fixtures, or interactions.

Sources. Keyframes are saved from both interaction events and perception events. Interaction keyframes include frames before, during, and after a manipulation, as well as analyzer-selected peak frames when an interaction reveals useful contents while perception-side keyframes capture new significant scene graph changes.

Trigger	Source	Purpose
interaction_pre	Manipulation	State before action
interaction_during	Manipulation	Mid-action evidence
interaction_post	Manipulation	State after action
interaction_peak	Analyzer	Informative reveal frame
scene_graph_delta	Perception	Significant graph update

Table 4: Sources of saved keyframes.

Metadata. Each keyframe stores metadata that makes it searchable and linkable to the scene graph: timestamp, camera, trigger source, parent interaction when applicable, linked scene graph nodes, robot pose, labels, caption, and image paths.

```
{
  "frame_id": "a1b2c3d4e5f6...",
  "timestamp": 172.4,
```

```

"session_id": "20260603_142210",
"camera": "base1",
"trigger": "interaction_peak",
"action": "OpenCabinet",
"parent_frame": "<pre-frame id or null>",
"scene_graph_nodes": ["cabinet_3", "banana_1"],
"robot_pose": [x, y, theta],
"tags": ["banana", "mug"],
"labels": [{"label": "cabinet interior", "source": "analyzer"},
            {"label": "banana", "source": "analyzer"}],
"caption": "...",
"image_path": "...",
"thumb_path": "..."
}

```

Duplicate filtering. To keep the keyframe memory compact, the system filters out blank, blurry, and near-duplicate frames before saving perception-triggered keyframes. Interaction-triggered keyframes are preserved even if visually similar, since pre-, during-, post-, and peak-interaction frames may encode outcome-relevant evidence.

B.3 VLM Retriever and Planner

The retriever selects task-relevant keyframes given the current task, scene graph, and keyframe meta-data. It first filters candidate frames using textual metadata and scene graph, then reranks candidate thumbnails with a VLM. The planner receives the retrieved frames alongside the current scene graph, current image, execution history, and list of motion primitives.

Retriever prompt. The full retrieval prompt is shown below. The no-scene-graph ablation removes the scene graph block and graph-node column.

```

You are a keyframe retrieval agent for a robot planner.

The robot has been given this task: "{goal}"
Here is the current scene graph (objects the planner knows about):
{scene_graph_summary}

Below is a catalog of stored keyframes. Each row: trigger, camera, action, the
scene-graph nodes it depicts, and any analyzer/instruction labels.
{catalog}    // idx | frame_id | trigger | camera | action | graph_nodes | labels

Choose up to {max_k} keyframes that would most help the planner. Consider: frames
showing objects/locations in the task; container interiors the planner must
reason about; spatial relationships (behind, inside, on top of). Return [] if none
help.

Return STRICT JSON ONLY:
{"chosen": [{"idx": <i>, "frame_id": "<first 12 chars>",
               "reason": "<one line on why this frame helps>"}]}

```

Planner prompt. We show the prompt for our full method (the scene_graph+kf configuration; the baseline ablations strip the corresponding scene-graph fields and rules). The planner is called repeatedly in closed loop: it returns a multi-step plan, the system executes only the first step, then re-queries with updated observations and execution history. The current camera image is provided alongside the text, and retrieved keyframes are appended as additional images captioned with their trigger, camera, and retrieval reason.

Simulation planner (RoboCasa kitchen).

```

You are a task planner for a PandaOmron mobile manipulation robot in a RoboCasa
kitchen. The robot has a Panda arm mounted on an Omron mobile base with an

```

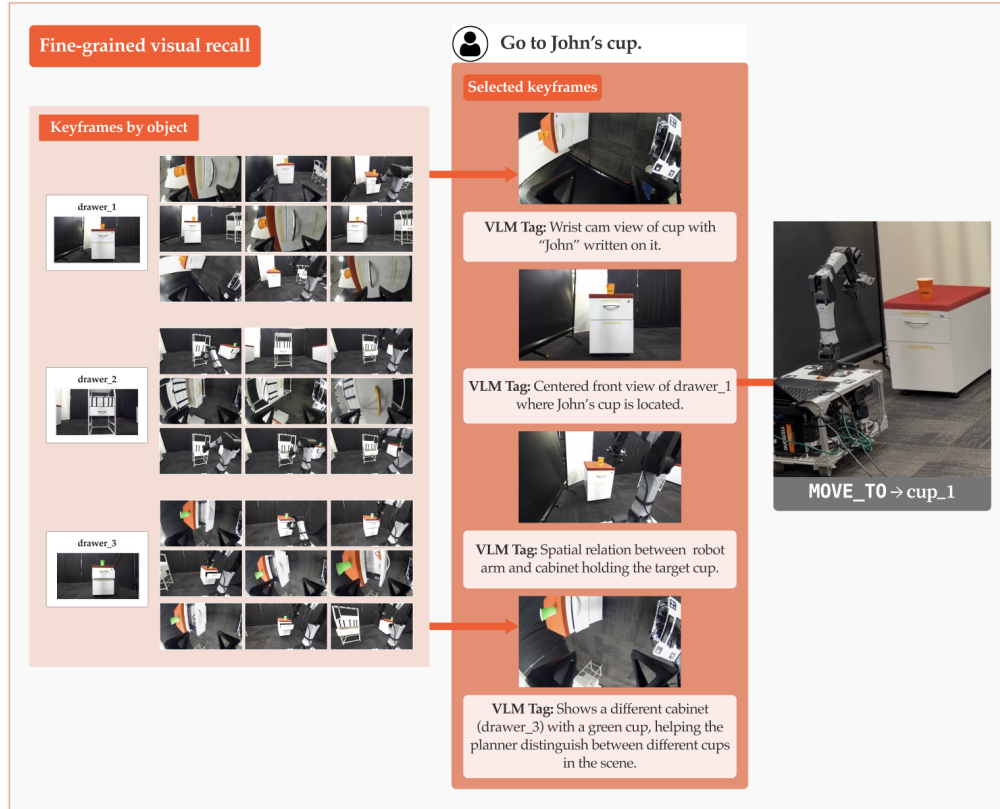


Figure 5: **VLM retriever effectively selects a task-relevant subset of keyframes for planning.** Given the scene graph and task description, the VLM retriever selects keyframes that are useful for the current goal and tags each one with a short reason. In this example, the task is to go to John's cup: the retrieved frames show both the orange cup on the left drawer and the green cup on the right drawer, allowing the planner to identify the orange cup labeled "John" as the correct target.

eye-in-hand wrist camera.

Goal: "{goal}"

{execution_history} // prior attempts in order, each with SUCCESS/FAILED status

Available primitives (you may ONLY use these):

{primitives} // name: description / Args / When

Kitchen fixtures in the scene (interactable objects the robot can navigate to): {fixtures}

Scene graph (objects with 3D world positions, interaction-informed properties, and inspection results including any "contents" found inside containers): {scene_graph}

You are also given an image showing the current view of the scene. Use both the scene graph data and the visual information to reason about spatial layout, obstacles, and plan feasibility.

Robot physical constraints:

- The Panda arm has ~0.85m reach from the base centre.
- The Omron base is ~0.55m wide - it cannot fit through gaps narrower than 0.6m.
- Use move_base_to with a fixture name (target_fixture) to navigate the robot in front of any fixture; the system computes the approach position and heading.

- Use `move_base_to` with explicit `target_pos [x, y, yaw]` for non-fixture locations.
- Use `move_arm_to` for fine EEF placement before manipulation tasks.
- Always `move_base_to` the target fixture BEFORE attempting `OpenCabinet`, `PickPlace`, or `Inspect` - the arm must be in range of the target.
- `Inspect` performs a wrist-camera sweep across an open fixture, writing object identifications into `scene_graph[fixture]["contents"]`. Decide goal satisfaction from these contents (and any retrieved memory frames showing the fixture).

Generate a step-by-step plan to achieve the goal. You will be called repeatedly in a closed loop: you produce a plan, the system executes the FIRST step, then calls you again with updated observations and execution history. So plan ahead, but know that you will get a chance to adapt after every step.

Rules:

1. ONLY use actions from the available primitives list.
2. For fixture-targeted primitives, use the exact fixture name from the fixtures list.
3. Order steps logically based on what the goal requires.
4. For objects that have properties (e.g. pickable, pushable, accessible, fullness, weight), leverage them to decide WHETHER to interact and which action to use.
5. Objects may have a `task_history` showing past attempts and their success/failure - use this to inform action choices. If an action failed on an object, do NOT retry a similar action on the same object - adapt your strategy.
6. Reference objects by their scene graph ID (e.g. `banana_10`, `ketchup_3`).
7. For `move_base_to`, provide `target_fixture` OR `target_pos` as `[x, y, yaw_radians]`. For `move_arm_to`, provide `target_pos` as `[x, y, z]`. For all other primitives, provide `target_fixture` as the fixture name string.
8. For `Inspect` steps, set `target_object` to the object label the goal asks to find (e.g. "banana"). Use null for other actions.
9. After opening a container (cabinet, fridge, dishwasher) with an `Open*` primitive, always plan a corresponding `Close*` step before moving on or finishing the plan.
10. The goal is achieved once its literal request is satisfied. Informational verbs (find / locate / identify / where-is / describe / open) are satisfied as soon as the relevant state is observable from `task_history`, scene graph, or current image - no further physical interaction is required. Navigation verbs are satisfied as soon as `task_history` contains ONE successful `move_base_to` on the target fixture. Action verbs (pick up / place / bring) require the literal physical action. If the goal asks a question, your 'assumptions' must contain the literal answer before returning an empty plan. If the goal is already achieved, return an EMPTY plan: `{"plan": [], "assumptions": ["Goal already achieved."], "warnings": []}`.

Return STRICT JSON ONLY:

```
{
  "plan": [
    {
      "step": 1,
      "action": "<primitive_name>",
      "target_fixture": "<fixture_name or null>",
      "target_pos": [x, y, z_or_yaw],
      "target_object": "<object label to find, or null>",
      "reasoning": "..."
    }
  ]
}
```

```

],
"assumptions": ["..."],
"warnings": ["..."]
}

```

Real-world planner.

You are a task planner for a mobile manipulation robot.

Goal: "{goal}"

{execution_history} // prior attempts in order, each with SUCCESS/FAILED status

Robot physical constraints:

- The robot base footprint is approximately 0.55m wide x 0.51m deep.
- The move_to action drives in a STRAIGHT LINE toward a target [x, y, z] position.
- The arm workspace is limited to +/-0.28m in the Y (left-right) direction from the base center.
- The arm can reach objects roughly 0.3-0.5m in front of the base (X direction).
- To interact with an object (open, close, peek), the robot must be within arm reach of it.

Scene graph (objects with 3D world positions, interaction-informed properties, and inspection results including any "contents" found inside containers):
{scene_graph}

You are also given an image showing the current view of the scene. Use both the scene graph data and the visual information to reason about spatial layout, obstacles, and plan feasibility.

Image overlays:

- Some images include yellow circles with scene-graph node IDs (e.g. 'drawer_1') drawn at the projected positions of current SG nodes, geometrically projected from the current scene graph using known camera calibration.
- When an overlay appears in an image, prefer it as the authoritative ID mapping for that pixel region. If a memory frame's text note mentions a different ID than the overlay shows, defer to the overlay.
- Overlays appear only on the calibrated base frames. Wrist-camera frames pass through unannotated - for those, use the frame's text note as your grounding signal.
- An image may have no overlays even when one was expected (robot pose at capture was unknown). In that case, ground using visual + scene-graph reasoning as usual.

Available primitive actions:

{primitives} // name: description, per registered primitive

Generate a step-by-step plan to achieve the goal. You will be called repeatedly in a closed loop: you produce a plan, the system executes the FIRST step, then calls you again with updated observations and execution history. So plan ahead, but know that you will get a chance to adapt after every step.

Rules:

1. For objects that are in the path, leverage their properties (e.g. pickable, pushable, accessible, fullness, weight) to decide WHETHER to interact and which action to use.
2. Before issuing an action, check the target object's current properties in the scene graph to avoid redundant commands. For example, if is_opened is already

true, consider whether you really need to re-issue an open command. You may still re-issue if the prior attempt only partially succeeded, but prefer skipping if the state already matches the goal.

3. Objects may have a task_history showing past attempts and their success/failure - use this to inform action choices, including whether to retry, adapt the approach, or try a different target.

A drawer is only interactable if it has a visible handle the robot can grasp. Before planning ANY actions on a drawer (including move_to), verify from the images that it has a handle. A drawer with no handle is not openable - treat it as if it does not exist.

If an open_drawer or close_drawer step fails, distinguish two cases: (a) the robot grasped the handle correctly but the drawer did not move (e.g. it is locked) - treat the entire drawer unit as non-openable and move on to a completely different drawer or fixture. Do NOT attempt to open a different drawer in the same unit; (b) the robot missed the handle or approached from a bad angle - retry the same action once, since visual pointing can produce a bad initial pose on the first attempt.

When a new goal implies the environment has changed since prior inspections (e.g. "a new item was added"), the 'contents' recorded in the scene graph may be STALE.

Do not assume prior contents are still accurate. Use the goal description, scene graph properties, and task_history to decide which objects are worth re-inspecting - just the ones that are plausible given what you know and the task description.

4. When deciding where to put an object, refer to the scene graph contents and keyframe images of container interiors. Consider what items are already inside each container and where the object best belongs.

5. The robot can ONLY interact (open, close, peek) with the object it last did move_to on. Find the LAST move_to in the most recent goal in the execution history - if it differs from your target, plan a move_to first.

6. Reference objects by their scene graph ID (e.g. green_cube_0, red_cube_1).

7. ONLY use actions from the available primitives list.

8. Each step must have a target_object that exists in the scene graph (use the exact ID).

9. After inspecting or exploring, consider actions that increase executability of future steps (e.g. close an upper drawer to access a lower one).

10. Goal completion depends on the verb type. Informational verbs (find / locate / identify / where-is / describe / open) are satisfied as soon as the relevant state is observable from task_history, the scene graph, or current image - no further physical interaction with the target is required. Navigation verbs (navigate / go to / drive to / approach) are satisfied as soon as task_history contains ONE move step on the target or on the scene graph node nearest to / containing the target - do NOT re-issue the same navigation. Action verbs (pick up / place / bring) require the literal physical action. If the goal asks a question (describe / name / list / how many / which), your 'reasoning' must contain the literal answer before you return an empty plan. If the goal is already achieved, return an

EMPTY plan: {"plan": [], "reasoning": ["Goal already achieved."], "warnings": []}.

11. Some primitives are localized by VLM pointing on a camera frame. For those steps,

you may include an optional 'pointing_description' field - a short visual phrase describing exactly WHERE on the target the action should engage. Use it when the scene_graph_id is ambiguous (e.g. one 'drawer_0' node represents a stack of multiple visible drawers). Examples: "the upper drawer handle", "the left cabinet door handle". Omit when the SG label alone is unambiguous.

Return STRICT JSON ONLY:

```
{
  "plan": [
    {
      "step": 1,
      "action": "<one of the available primitives>",
      "target_object": "<scene_graph_id>",
      "pointing_description": "<optional visual phrase; omit if unneeded>",
      "grounded_by_keyframe_idx": <optional int; 1-indexed memory frame used to
                                identify target_object; omit when identified
                                from the current scene only>,
      "reasoning": "..."
    }
  ],
  "reasoning": ["..."],
  "warnings": ["..."]
}
```

Projecting scene graph nodes into images. In real-world experiments, we use Set-of-Mark annotations [49] to spatially ground the planner on calibrated base-camera images. For each visible scene graph node, we project its 3D position into the image and overlay a circle with the object label. Wrist-camera keyframes are left unannotated and are grounded through their captions and linked scene graph metadata.

C Simulation Scenario Details

C.1 Scenario Descriptions

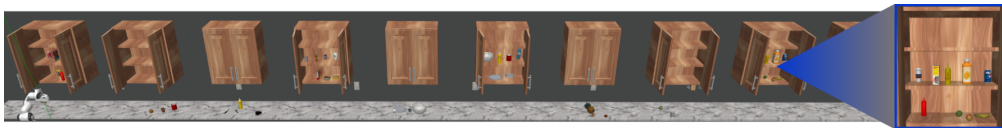


Figure 6: **25-task Long Horizon.** The 25-task sequence takes place in a ten-cabinet kitchen with three locked cabinets and **more than 60 objects and fixtures to track**. Countertop objects provide distractors and visual references. The inset shows the contents and spatial arrangement of one cabinet.

25-task Long Horizon. We evaluate an ordered sequence of 25 household tasks in a 10-cabinet RoboCasa365 kitchen containing more than 60 objects and fixtures. Three cabinets are locked, and objects on the counters provide distractors and visual references. An initial search for a teapot in the final cabinet encourages exploration; subsequent tasks reuse earlier observations and interaction outcomes without resetting memory or the environment. The sequence spans four task families:

- **Finding (9 tasks).** Locate objects using remembered locations, neighboring items, or accessibility. Examples include distinguishing a stored mug from one on a counter, identifying ketchup by its surrounding condiments or fruit, and finding the bottle with clear space for grasping.

- **Matching (9 tasks).** Distinguish three visual variants within each of seven object categories, using either a specified identity or a countertop reference. Similar colors and category labels make branding and appearance important. Targets are also placed in unexpected locations, such as Corn Flakes in the fruit cabinet, to discourage shortcuts based on cabinet categories.
- **Manipulation (3 tasks).** Select where bread, a mug, or a bowl belongs using remembered cabinet contents. For example, bread belongs with the pantry’s dry goods even though no cabinet already contains bread. The robot selects a destination without transporting the object.
- **Memory queries (4 tasks).** Recall previously observed contents and spatial relationships, such as what was stored with the mayonnaise or where the croissant near the pineapple was located. These questions can be answered from existing memory without further exploration.

Locked-and-Unlocked Cabinets. Three cabinets: cab_1 (far-left) is locked and empty, cab_2 (middle) holds the banana among fruit distractors, and cab_3 (far-right) holds the ketchup but is never observed during the first instruction. Spawning in front of cab_1, the robot is first instructed “Look into upper cabinets and find the banana.” Its first attempt to open cab_1 fails (locked), after which it finds the banana in cab_2, recording that cab_1 is locked and cab_2 lacks ketchup. Returned to the first cabinet, it is then instructed “Find the ketchup.” Success requires navigating to the unexplored cab_3 while skipping the locked cab_1 and the ketchup-free cab_2; re-attempting the locked cabinet is considered a failure. Figure 7 compares MessyMem against **SG+KF**, showing the utility of the interaction analyzer. By reasoning about its first failed opening, MessyMem records the left cabinet as locked and uses this memory on the next task when finding a new item. In contrast, **SG+KF** attempts to re-open the first drawer, failing the task.

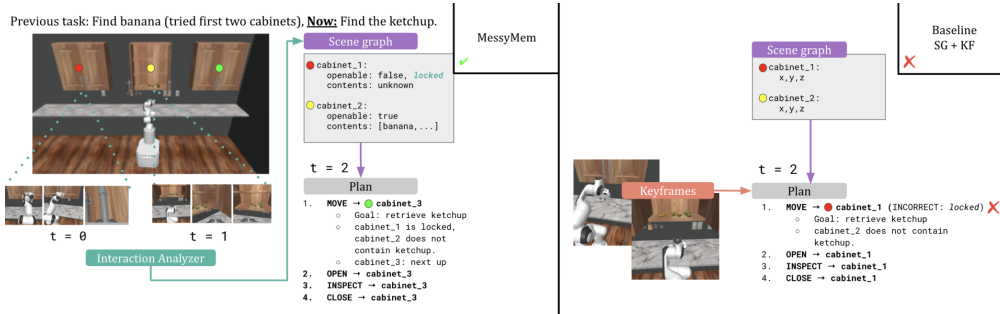


Figure 7: **Locked-and-Unlocked Cabinets.** This scenario highlights the role of the interaction analyzer. By reasoning over the interaction collage, MessyMem correctly determines the first cabinet as locked and stores this in the scene graph. On the next task, the robot uses this memory, together with the remembered contents of the second cabinet, to skip both and move to the remaining cabinet.

Clutter-Aware Pick. Given two cabinets, both of which contain a ketchup bottle, but in cab_1 it sits alone with clear side access (five other items packed to one side), whereas in cab_2 it is wedged among two tightly-clustered items. The cluttered cabinet holds *fewer* items overall, so item counts are anti-correlated with accessibility. The robot is first instructed “Look in the upper cabinets and find the mustard.” (mustard is in cab_2), opening both cabinets and seeding memory with interior keyframes. It is then instructed “Pick up the ketchup from a cabinet and place it on the counter.”, ending in front of the already-open cab_2. By choosing based solely on proximity, the robot will attempt the wrong, cluttered cabinet. A scene-graph only (**SG**) method cannot confidently determine the correct cabinet as it only records the broad category of “ketchup”. Success requires comparing the two interiors from the stored keyframes and picking from the less-cluttered cab_1, testing whether keyframe memory supplies spatial information beyond semantic labels and positions. Figure 8 compares MessyMem against **SG+IA**, highlighting the impact of the keyframe memory. In this

scenario, when needing to pick the ketchup, MessyMem enables querying relevant keyframes for the VLM to decide which cabinet is more accessible and less cluttered for picking. In contrast, **SG+IA** solely relies on how many objects there are in a cabinet, which is an insufficient signal, and therefore picks up the ketchup that is cluttered amongst other condiments.

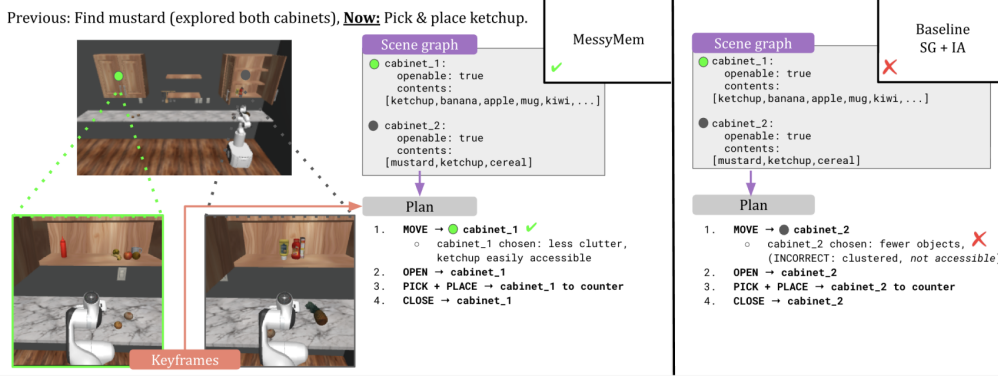


Figure 8: **Clutter-Aware Pick.** This scenario demonstrates the importance of keyframe memory. The robot has to look back at prior visual observations to compare fine-grained details such as clutter, free space, and object layout. The planner is able to query the relevant keyframes to determine picking from which cabinet is more accessible and less likely to disturb nearby objects.

C.2 Baselines

All methods use the same environment layouts, perception front-end, primitive skills, planner model, and task-success checks unless otherwise noted. We compare against both internal ablations of MessyMem and external memory representations.

Internal ablations. Our ablations selectively remove components of MessyMem while keeping the remaining system fixed.

- **MessyMem (SG+IA+KF).** Our full method combines a persistent 3D scene graph, interaction-analyzer updates, and linked keyframe memory.
- **SG+IA.** Scene graph memory with interaction-derived updates, but without keyframes. This tests whether structured interaction outcomes alone are sufficient without access to fine-grained visual evidence.
- **SG+KF.** Scene graph memory with linked keyframes, but without interaction-analyzer updates. This tests whether visual recall can compensate for missing interaction-derived properties and outcomes.
- **SG.** A persistent scene graph containing object labels and positions, but no interaction-derived updates or keyframe memory. This tests the value of a spatially grounded memory without either interaction semantics or visual recall.

RoboEXP-ACSG. We adapt RoboEXP’s Action-Conditioned Scene Graph (ACSG) [13] from its released implementation. RoboEXP is designed for interactive exploration, where actions reveal hidden scene structure that is incorporated into a relational graph containing object identities, geometry, and coarse relations such as *inside*, *on*, and *under*. We use the ACSG as a drop-in replacement for MessyMem’s memory while keeping perception, planning, primitives, and evaluation fixed. Rather than running RoboEXP’s exploration policy end-to-end, we adapt its graph updates to our shared stack and allow the graph to persist across sequential tasks and be queried by the same closed-loop planner as MessyMem, strengthening the baseline beyond its original exploration-focused setting. Unlike MessyMem, RoboEXP-ACSG does not retain linked visual keyframes or richer interaction-derived properties and outcomes, such as whether a fixture was discovered to be locked.

MemER-style. We implement a MemER-style visual memory following the high-level memory selection procedure of Sridhar et al [6]. We use the term *MemER-style* because we reproduce its memory mechanism without fine-tuning or reproducing its original low-level policy. At each update, a VLM identifies temporally informative frames; nearby selections are clustered along the time axis, and representative frames are retained in a bounded visual memory. We follow the original integer hyperparameters with a memory length of 8, recent window of 8, and temporal merge distance of 5.

MemER was originally developed in a tabletop manipulation setting, where the relevant workspace remains largely within view. Its memory therefore consists of selected past images without a persistent spatial index associating those images with navigable scene entities. In our mobile-manipulation setting, objects and fixtures frequently leave the robot’s current view, so generated subtasks must be grounded from the current observation alone. We use the same VLM and primitive execution stack as the other methods so that differences primarily reflect the memory representation rather than the underlying model or controller.

MemER+Fixtures. We additionally evaluate **MemER+Fixtures**, which uses the same visual memory mechanism as MemER-style but additionally provides the planner with the identities and world positions of the kitchen fixtures. This gives the method an explicit spatial vocabulary for referring to and navigating toward out-of-view destinations while leaving its visual memory selection unchanged. The comparison between MemER-style and MemER+Fixtures measures how much explicit spatial grounding helps when adapting their method to mobile manipulation.

D Real-World Scenario Details

D.1 Task Descriptions

Office Drawer Search. The robot searches three office drawer units where the left drawer is locked, the middle and right are unlocked. The scissors are in the right drawer (with a green cup on top), a banana is in the middle drawer, and an orange cup labeled “John” sits atop the locked left drawer; before the final instruction, a gaming controller is placed in the right drawer. The robot receives three sequential instructions: “*Find me scissors.*”, then “*Move to John’s cup.*”, then “*Since you finished your last task, my friend just placed a gaming controller into one of the unlocked drawers. Help me find it.*” Across these it must remember which drawer failed to open (locked), what contents were already observed, and which locations remain plausible for the next target.

Sock Pairing. Three fixtures are arranged left-to-right, two drawers and a chair, with socks in each drawer and one lone sock placed on the chair. The lone sock’s distinguishing attribute (color or pattern) and which drawer holds its match are swapped and randomized between trials, so the answer cannot be memorized and must be inferred from observation. The robot receives three sequential instructions: “*Move to each drawer unit, open it, and peek inside to observe the contents.*”, then “*Move to the chair.*”, then “*Pick up the lone sock from the chair and place it into the drawer with its matching pair.*” Success requires visual memory. The planner must recall which drawer contains the sock matching the lone sock, a distinction of color or pattern rather than object category alone.